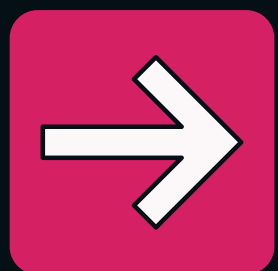


ANGULAR CONTROL FLOW

What's new with `@if`,
`@for` and `@switch` in
Angular?



01 SYNTAX COMPARISON

Angular 18 introduces a new control flow syntax:

Angular 17

Angular 18

*ngIf → @if

*ngFor → @for

*ngSwitch → @switch

*They simplify template logic without the need for ***ngIf** or ***ngFor**.*



02 BEFORE VS AFTER

→ Old way (Angular 17):



```
*ngIf="condition"
```

```
*ngFor="let item of items"
```

```
[ngSwitch]="value"
```

→ New way (Angular 18):



```
@if (condition) { ... }
```

```
@for (item of items) { ... }
```

```
@switch (value) {  
  @case (...) { ... }  
  @default { ... }  
}
```

03 WHY THIS MATTERS?

You might be wondering: why switch to this new structure?



More Intuitive

No more asterisks or structural directive quirks



Safer Templates

TS-aware blocks, IDE support, better autocomplete.



Faster Rendering

Improved hydration and fine-grained reactivity (with Signals!).

04 REAL USAGE

Simpler than ng-template + ng-container madness!



```
@for (student of students; track student.id) {  
  <app-student [data]="student" />  
  
} @empty {  
  <p>No todos yet.</p>  
  
}
```

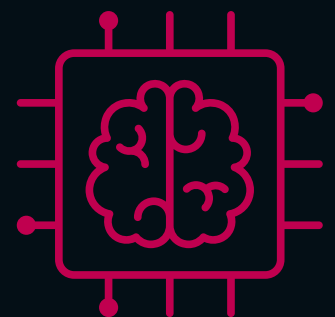
This is cleaner, more readable, and type-safe.

05 WHEN TO USE CONTROL FLOW?

This new syntax isn't just syntactic sugar, it's a real productivity booster.

- ✓ Always, from Angular 18+
- ✓ For cleaner code in components
- ✓ Combined with Signals for reactive templates

Angular is finally catching up with other frameworks like SolidJS and Svelte, and it's exciting.



LOVED IT?
DROP YOUR THOUGHTS
OR TIPS IN THE
COMMENTS!

